

GUÍA PASO A PASO – CONEXIÓN BACKEND CON BASE DE DATOS EN RAILWAY

Esta guía describe paso a paso la construcción de una aplicación Backend que evoluciona desde una API simulada en local hasta una API pública conectada a una base de datos MySQL. Aquí mostramos el proceso completo para conectar un Backend en Node.js con una base de datos localizada en un servidor externo. En este caso Railway.com ofrece la posibilidad de publicar una API accesible desde Internet.

ETAPA 1 – CREACIÓN DE LA BASE DE DATOS EN RAILWAY

En esta etapa se crea la base de datos en Railway y se obtienen las variables de entorno necesarias para conectarse desde herramientas externas y desde el Backend.

1. Creación del proyecto y servicio MySQL

Ingresa a Railway y crear un nuevo Proyecto, luego agregar el servicio de base de datos MySQL. Railway. Este generará automáticamente la nueva Base de datos , junto con las credenciales de conexión.

El ingreso a Railway lo podrás realizar utilizando tu conexión con GitHub.

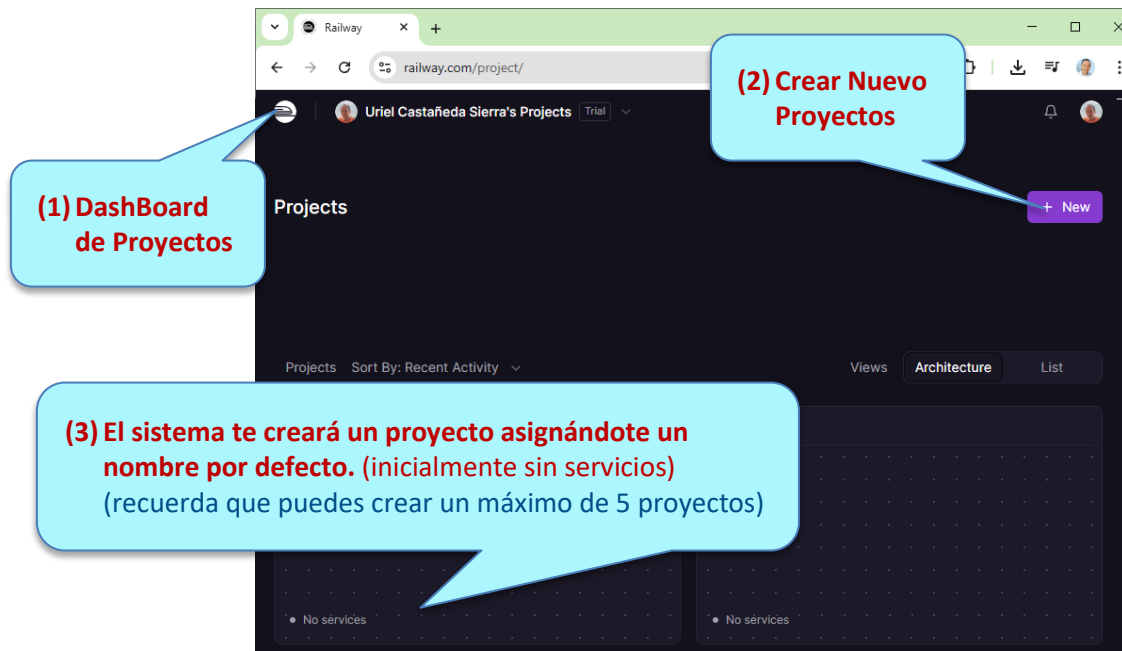
Una vez hayas ingresado observarás la pantalla del dashboard de proyectos. Inicialmente no tendrás proyectos. ***IMPORTANTE*** Tener en cuenta que si estás trabajando con acceso gratuito, el Sistema te restringirá algunas cosas, entre ellas:

- Solo podrás crear un máximo de 5 proyectos
- No todas las funcionalidades te serán habilitadas.
- No podrás emitir instrucciones SQL para las bases de datos. Sin embargo en forma alternativa podrás hacerlo utilizando una herramienta para administración de bases de datos. Para esto se sugiere utilizar [HeidiSQL](#), que es un software libre, de muy fácil utilización y además es portable.

A continuación el paso a paso para la creación de tu Proyecto, en donde incluirás la base de datos y el componente de tu Back-End.

1.1. Creación del Proyecto en RailWay

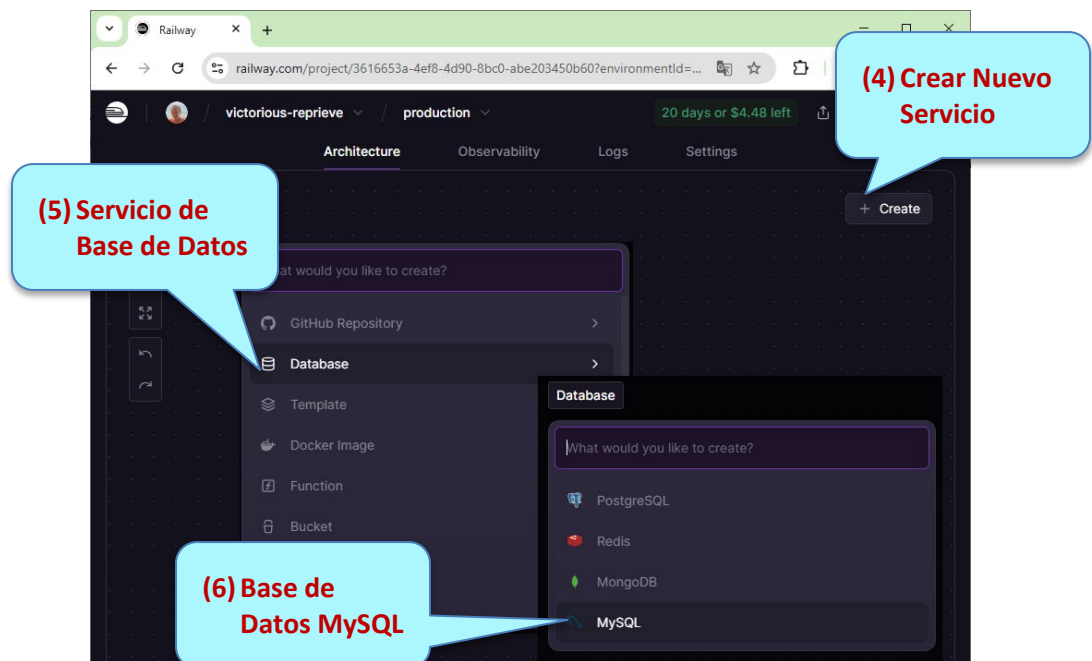
Después de crear tu usuario vinculado con GitHub, debes asegurar estar ubicado en el dashboard de proyectos (1), en donde se visualiza inicialmente el listado de proyectos vacío. Luego (2) seleccionar el botón [+ New]. (3) El Sistema creará el proyecto dejándolo listo para adicionar servicios.



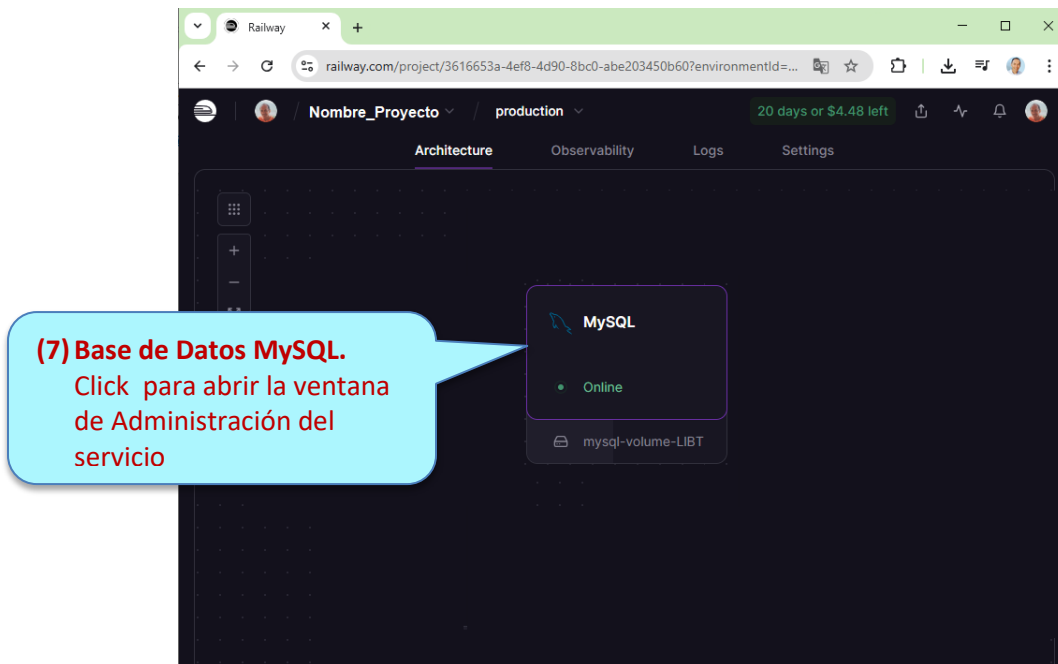
1.2. Creación del servicio de Base de Datos

Una vez creado el Proyecto, se procede con la CREACIÓN DEL SERVICIO DE BASE DE DATOS. Para ello, (4) marcar click en el botón [+ Create]. Allí el Sistema despliega el listado de servicios posibles.

(5) Seleccionar **Database** y luego (6) seleccionar **MySQL**.



Una vez seleccionada la opción de BASE DE DATOS, el Sistema iniciará el proceso de creación de components y (7) finalmente mostrará un cuadro o botón representativo de la B.D. MySQL, indicando que se encuentra **on-line**. Haciendo click en este botón podrá observar la Ventana de administración de este servicio.



1.3. Obtención de variables de entorno para conexión con la B.D. en Railway

Después de crear la base de datos MySQL en Railway, se deben copiar las variables de entorno mostradas en la sección de conexión. Estas variables permiten conectarse tanto desde **HeidiSQL** como desde el **Backend**.

```
MYSQLHOST=...  
MYSQLPORT=...  
MYSQLDATABASE=...  
MYSQLUSER=...  
MYSQLPASSWORD=....
```

Para obtener estos datos, en la Ventana de Administración del servicio, ejecute los siguientes pasos:

(8) Seleccionar CONNECT
Aquí se abrirá la ventana de parámetros de conexión

(9) Seleccionar la pestaña Public NetWork. De aquí tomaremos los principales datos de conexión.

(10) MYSQLHOST. Nombre del Host o servidor en Internet

(11) MYSQLPORT. Nombre o número del puerto de conexión

(12) MYSQLDATABASE. Nombre del base de datos

(13) MYSQLUSER. Nombre del usuario con permiso de conexión

The screenshot shows the 'Connect to MySQL' dialog with the 'Public Network' tab selected. The 'Connection URL' field contains the text: `mysql://root:*****@eeeeee.proxy.rlwy.net:56569/railway`. The 'Raw `mysql` command' field shows: `mysql -h eeeee.proxy.rlwy.net -u root -p ***** --port :56569, --protocol=TCP rail_`. The 'Railway CLI `connect` command' field shows: `railway connect MySQL`.

Solamente falta obtener el dato de la variable **MYSQLPASSWORD**. Esta puede ser obtenida accediendo a la pestaña "Variables" de la Ventana de Administración del servicio de Base de datos, así:

(15) Seleccionar la pestaña Variables. Aquí Buscamos la variable **MYSQL_ROOT_PASSWORD**.

(14) MYSQLPASSWORD. Aquí podemos visualizar el **password** de conexión y copiar su contenido.

The screenshot shows the 'Variables' tab in the MySQL interface. It lists 9 Service Variables. The variable **MYSQL_ROOT_PASSWORD** is highlighted, and its value is masked with asterisks. A 'Show value' button is visible next to it.

Finalmente el conteido de las variables debe quedar algo similar al siguiente:

- MYSQLHOST=eeeeee.proxy.rlwy.net
- MYSQLPORT=56949
- MYSQLUSER=root
- MYSQLPASSWORD=ndltVWZArndltVWftAdndltVWwlXKhhNJh
- MYSQLDATABASE=railway

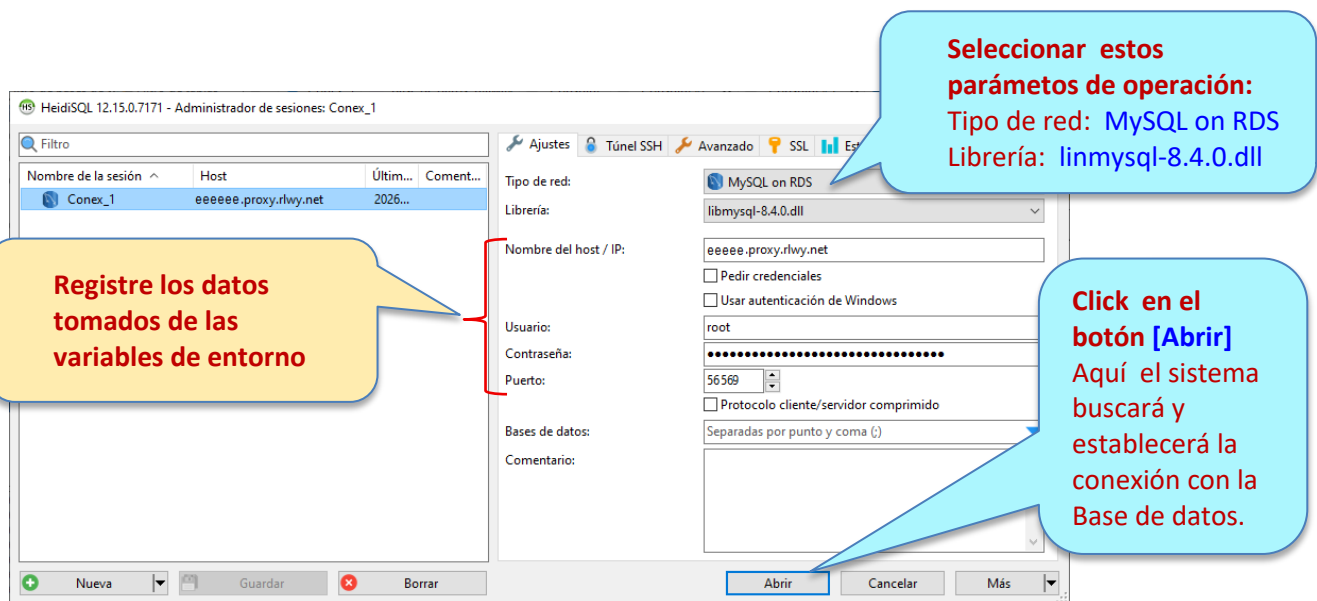
2. Conexión con la Base de datos usando un Software Administrador

El servidor de RailWay no prove servicios de administgración de la base de datos. Esta debemos realizarla utilizando un software DBMS (Sistema administrador de bases de datos) especializado y para ello se sugiere la utilización de **HeidiSQL**, este es un software libre que permite conectarse a diferentes motores de bases de datos.

Descarga oficial: <https://www.heidisql.com/download.php>

2.1. Conexión de la Base de datos en el Servidor RailWay

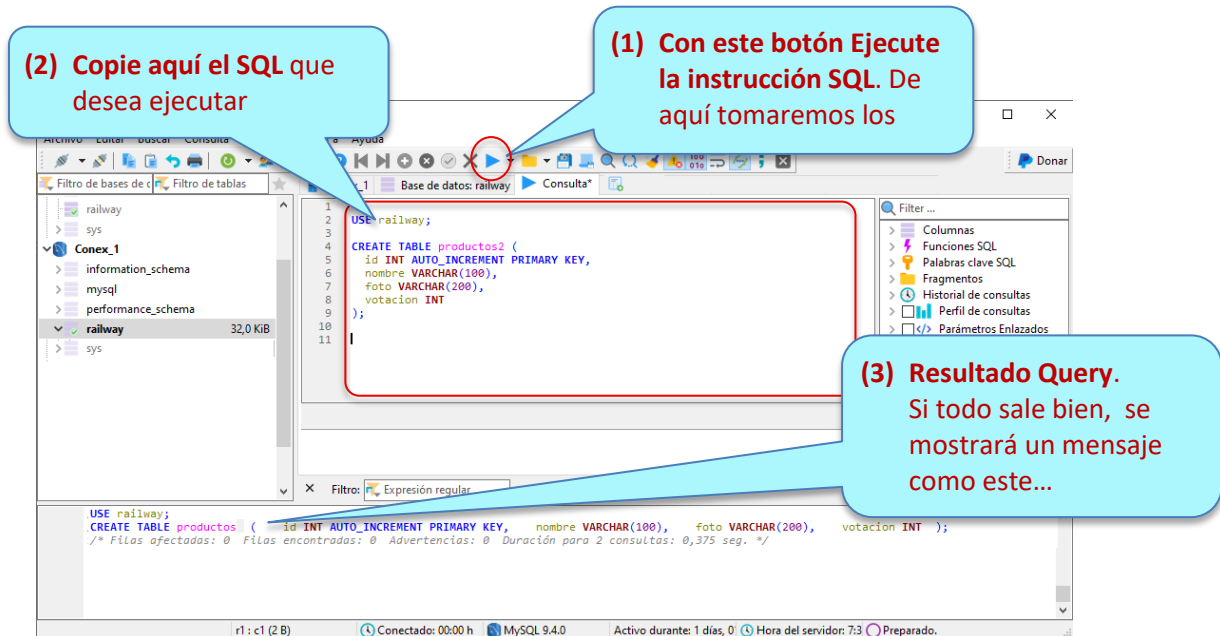
Al ejecutar el DBMS **HeidiSQL** se visualiza la Ventana de parámetros de conexión. Allí deberá ingresar los datos tomados de las variables de entorno de la base de datos, recientemente tomadas de RailWay.com o del servidor elegido.



Si todo sale bien, el DBMS **HeidiSQL** establecerá la conexión con el servidor y quedará habilitado para gestionar la administración y realizar operaciones sobre la base de datos. A través de la interface de administración se podrá editar y lanzar la ejecución de los Querys en SQL para crear la tabla de productos e insertar los registros de esta.

2.2. Creación y llenado de la tabla de productos:

Recordemos que la nueva base de datos creada por RailWay.com aun no cuenta con tablas. Éstas las crearemos mediante la ejecución de Querys (consultas con código SQL) lanzados desde HeidiSQL, así:



SQL para creación de la tabla productos

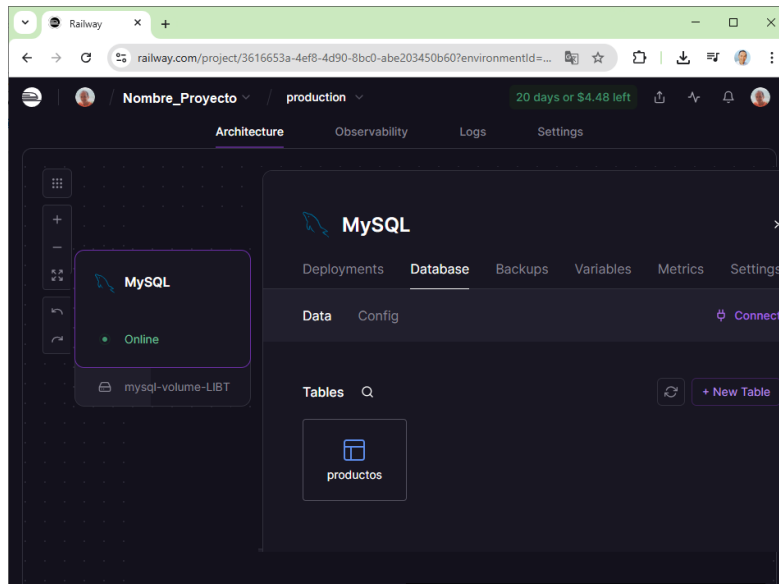
```
USE railway;
```

```
CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  foto VARCHAR(100),
  votacion INT
);
```

SQL para Inserción de registros de prueba

```
INSERT INTO productos (nombre, foto, votacion) VALUES
('Revitalift','revitalift.avif',120),
('Elseve','elseve.jpg',98),
('Infallible','infallible.jfif',150),
('Elvive','elvive.avif',87),
('True Match','truematch.webp',110),
('Preference','preference.jpg',65),
('Men Expert','menexpert.jpg',45),
('Age Perfect','ageperfect.webp',72),
('Casting Crème','castingcreme.webp',90),
('Studio Line','studioline.jpg',30);
```

Posterior a la ejecución de los SQLs para creación de la tabla de productos y la inserción de registros de prueba, ya podremos confirmar que la tabla **productos** ya sea visible en la pantalla de administración de la base de datos de **RailWay**.



2.3. Prueba de la instrucción SQL para generar el listado de productos

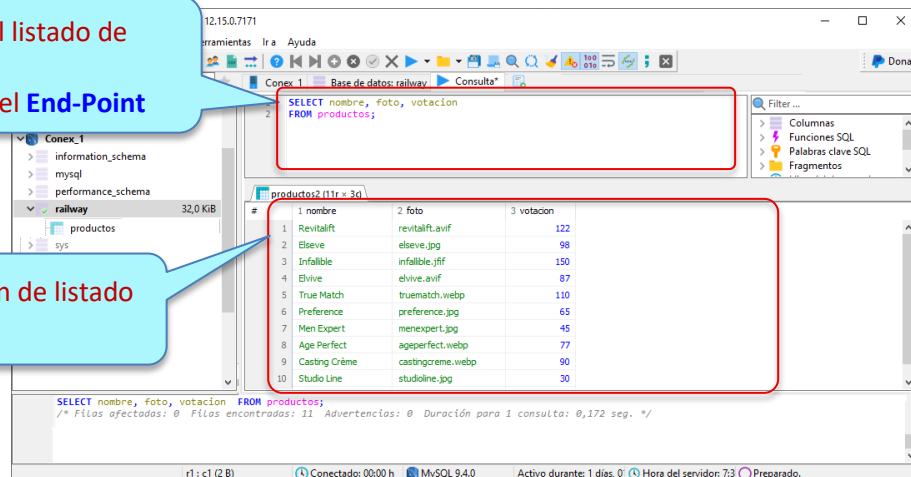
A través de HeidiSQL realizaremos la prueba de la instrucción SQL que utilizaremos más adelante en la construcción del Código que permitirá la actuación del EndPoint en Nuestro aplicativo BackEnd.

Registre las siguientes instrucciones en la ventana SQL del DBMS HeidiSQL.

```
SELECT nombre, foto, votacion FROM productos;
```

(4) SQL que genera el listado de productos.
Este será el SQL del End-Point

(5) Confirmación de listado resultante



ETAPA 2 – CREACIÓN DE LA API LOCAL CONECTADA A RAILWAY

3. Adaptación de la nueva API en la nueva versión del BackEnd.

Para el desarrollo de este proyecto tomamos como punto de partida una de las copias funcionales del Backend. Correspondiente a la versión previa que simulaba la API con un arreglo de objetos. Sobre esa base se realizan las modificaciones necesarias para conectarlo a MySQL.

3.1 Copia del proyecto base

Duplicar la carpeta del Backend funcional para conservar la versión anterior como respaldo.

3.2 Depuración de dependencias innecesarias

```
# Eliminar paquetes no utilizados (ejemplo: PostgreSQL)
npm uninstall pg
```

3.3 Instalación del cliente MySQL y dotenv

```
# Instalar dependencias necesarias
npm install mysql2 dotenv
```

3.4 Creación del archivo que configura variables de entorno (.env)

IMPORTANTE Recordar que este archivo **NO se debe subir a GitHub** que en el se exponen datos sensibles como el password y este quedaría visible al público para quienes descarguen el código disponible en el repositorio público de GitHub.

```
01 MYSQLHOST=eeeeeee.proxy.rlwy.net
02 MYSQLPORT=56569
03 MYSQLUSER=root
04 MYSQLPASSWORD= ndItVWZArndItVWftAdndItVWw1XKhhNJh
05 MYSQLDATABASE=railway
06 PORT=3000
```

3.5 Conexión a la base de datos MySQL (src/config/db.js)

Antes de editar el nuevo archivo, debemos crear la carpeta **src/config**.
Dentro de esta se creará el nuevo archivo **db.js**.

```
01 import mysql from 'mysql2/promise';
02 import dotenv from 'dotenv';
03
04 // Cargar variables de entorno desde el archivo .env
05 dotenv.config();
06
07 // Crear pool de conexiones reutilizable
08 export const pool = mysql.createPool({
09   host: process.env.MYSQLHOST,
10   port: process.env.MYSQLPORT,
11   user: process.env.MYSQLUSER,
12   password: process.env.MYSQLPASSWORD,
13   database: process.env.MYSQLDATABASE,
14   ssl: { rejectUnauthorized: false } // ← obligatorio en Railway
15 });
```


3.6. Endpoint para obtener productos desde MySQL (`src/routes/productos.db.js`)

```
01 import { Router } from 'express';
02 import { pool } from '../config/db.js';
03
04 const router = Router();
05
06 // GET /api/productos-db
07 router.get('/', async (req, res) => {
08   try {
09     // ---- Aquí se lanza el Query con las instrucciones SQL
10     //       que permiten obtener el listado de todos los productos
11     const [rows] = await pool.query(
12       'SELECT nombre, foto, votacion FROM productos'
13     );
14
15     // ---- Emite la respuesta convirtiendo y entregando los datos en formato json
16     res.json(rows);
17   } catch (error) {
18     console.error(error);
19     res.status(500).json({ error: 'Error al consultar MySQL Railway' });
20   }
21 });
22
23 export default router;
```

3.6. Conectar el nuevo endpoint en la aplicación principal (`app.js`)

Se deben registrar dos nuevas líneas de código:

- una para importar la ruta de la nueva API
- La 2ª para Registrar en nuevo end-point

El código completo de `app.js` quedaría así:

```
01 // =====
02 // =   PRIMER SERVIDOR EXPRESS   =
03 // =====
04
05 import express from 'express';
06 import cors from 'cors';
07
08 import path from 'path';
09 import { fileURLToPath } from 'url';
10
11
12 // Para conectar las rutas al servidor
13 import productosRoutes from './routes/productos.routes.js';
14 import productosDbRoutes from './routes/productos.db.routes.js';
15
16 const app = express();
17
18 // Habilita CORS para permitir que el Front consuma las APIs
19 app.use(cors()); // ← Añade cabeceras CORS automáticamente
20 app.use(express.json()); // ← Permite recibir JSON en el body (para futuros POST)
```

```

21
22
23 // ---- SERVIR ARCHIVOS ESTÁTICOS ----
24 // Esto permite acceder a /imagenes/productos/archivo.jpg desde el navegador.
25 // Ej: http://localhost:3000/imagenes/productos/elseve.jpg
26 const __filename = fileURLToPath(import.meta.url);
27 const __dirname = path.dirname(__filename);
28 app.use(express.static(path.resolve(__dirname, '../public')));
29 // -----
30
31 // Todas las rutas del servidor se conectan en este prefijo
32 // Rutas de la API (prefijo /api/productos)
33 app.use('/api/productos', productosRoutes);
34
35 app.use('/api/productos-db', productosDbRoutes);
36
37
38 export default app;
39
40 /* ==== EXPLICACIÓN ====
41 express()
42 ➡ Crea la app servidor
43
44 app.use(cors())
45 ➡ Permite que otros orígenes (frontend) accedan
46 (Los navegadores respetan estas cabeceras CORS)
47
48 app.use(express.json())
49 ➡ Permite recibir datos JSON en futuras rutas POST/PUT
50
51 app.use('/api/productos', productosRoutes);
52 ➡ Todas las rutas definidas en productos.routes.js
53 "viven" bajo /api/productos
54 */

```

ETAPA 3 – DESPLIEGUE DEL BACKEND EN RAILWAY Y API PÚBLICA

Finalmente, el Backend se publica en Railway para que la API pueda ser consumida desde cualquier aplicación en Internet.

4.1 Subida del proyecto a GitHub

Railway despliega aplicaciones conectadas a repositorios GitHub, por lo que el Backend debe estar versionado y disponible en línea.

4.2 Configuración de variables de entorno en Railway

Las mismas variables del archivo .env deben configurarse en el panel de Railway. Esto permite que la aplicación en producción se conecte correctamente a la base de datos.

4.3 Prueba de la API pública

`https://tu-backend.up.railway.app/api/productos-db`

Si la URL retorna un arreglo JSON con los productos, el despliegue fue exitoso.